Testing Python with pytest

Assertions, fixtures, and tests for language tools

CS413

October 8, 2026

Testing as an executable specification

- A test describes one observable behavior and checks it automatically.
- Example: looking up a missing variable should raise a particular exception.
- A failing test gives a counterexample to the claimed behavior.
- Passing tests provide evidence for the chosen cases, not a proof for all inputs.

Today's goals

- Write and run tests with ordinary Python assertions.
- Choose boundary cases and express them with parametrization.
- Use fixtures to control inputs, resources, and side effects.
- Add regression tests for a small language implementation.

A small project

```
pytest_demo/
    stack.py
    test_stack.py
    test_resources.py
    conftest.py
    expr.py
    test_expr.py
```

Run commands from `examples/pytest_demo/`.

```
python3 -m venv .venv
source .venv/bin/activate  # POSIX shell
python -m pip install pytest
python -m pytest -q
```

The behavior we want to test

```python
# stack.py
class Stack:
    def __init__(self):
        self._items = []

    def push(self, value):
        self._items.append(value)

    def pop(self):
        if not self._items:
            raise IndexError("empty stack")
        return self._items.pop()
```

Contract: last in, first out. An empty pop raises `IndexError`.

A first test

```python
# test_stack.py
from stack import Stack

def test_pop_returns_last_pushed():
    stack = Stack()           # arrange
    stack.push("x")
    stack.push("y")
    result = stack.pop()      # act
    assert result == "y"      # assert
```

The test name describes the behavior. A failing assertion fails the test.

Discovery and execution

- By default, pytest searches for `test_*.py` and `*_test.py`.
- Test functions start with `test_`. Plain functions are enough.
- pytest imports test modules, collects tests, then runs them.
- Keep demonstrations out of module-level code to avoid import side effects.

```
python -m pytest --collect-only -q
```

Reading a failure

Suppose `pop()` incorrectly uses `self._items.pop(0)`.

```
>       assert result == "y"
E       AssertionError: assert 'x' == 'y'

FAILED test_stack.py::test_pop_returns_last_pushed
```

- The node ID identifies the file and test function.
- Read the actual and expected values, then inspect the contract.
- An exception during fixture setup is reported as an error.

Running a focused set of tests

```
python -m pytest -q
python -m pytest test_stack.py -v
python -m pytest test_stack.py::test_empty_pop
python -m pytest -k "pop and not empty"
python -m pytest -x
python -m pytest --lf
```

- `-k` selects by a name expression. `-x` stops on the first failure.
- `-lf` reruns previously failing tests using the local cache.
- During debugging, run a small selection. Before finishing, run the suite.

Assertions and the test oracle

```
def test_pop_removes_an_item():
    stack = Stack()
    stack.push("x")
    stack.push("y")
    assert stack.pop() == "y"
    assert stack.pop() == "x"
```

- The oracle is how the test decides the expected result.
- Choose expectations from the specification, independently of the implementation.
- Checking only that a result exists can miss an incorrect result.

Expected exceptions

```
import pytest

def test_empty_pop():
    stack = Stack()
    with pytest.raises(IndexError, match="empty stack"):
        stack.pop()
```

- No exception, or a different exception type, fails this test.
- `match` is a regular expression search on the exception message.
- Keep the block narrow. Put assertions about later behavior after it.

Several cases, one test

```
@pytest.mark.parametrize("values", [
    [],
    [7],
    [2, 5, 2],
], ids=["empty", "singleton", "duplicates"])
def test_lifo(values):
    stack = Stack()
    for value in values:
        stack.push(value)
    actual = [stack.pop() for _ in values]
    assert actual == list(reversed(values))
```

pytest collects a separate test for each case, with its own result.

Choosing cases that expose mistakes

- Empty input exercises a boundary or error condition.
- A singleton checks the smallest successful operation.
- Distinct values reveal ordering mistakes. Duplicates reveal lost elements.
- After pushing and popping everything, another pop should fail.
- A regression test reproduces a specific bug before the fix and passes after it.

Fixtures supply test inputs

```
# conftest.py
import pytest
from stack import Stack

@pytest.fixture
def stack():
    return Stack()

# test_stack.py
def test_fixture_stack(stack):
    stack.push(7)
    assert stack.pop() == 7
```

pytest resolves the argument name `stack` and supplies the fixture value.

Fixture lifetime and sharing

- The default scope is `function`: each test gets a fresh stack.
- A fixture may request another fixture through its own arguments.
- Put shared fixtures in `conftest.py`. Tests do not import that file.
- Wider scopes include `class`, `module`, `package`, and `session`.
- A mutable object shared across tests can make results depend on execution order.

Cleanup with a yield fixture

```
@pytest.fixture
def output_file(tmp_path):
    stream = (tmp_path / "out.txt").open("w+")
    yield stream
    stream.close()

def test_output_file(output_file):
    output_file.write("hello")
    output_file.seek(0)
    assert output_file.read() == "hello"
```

Setup runs before `yield`. Cleanup runs afterward, even if the test fails.
If setup fails before reaching `yield`, that trailing cleanup does not run.

Temporary files with tmp_path

```
def test_save_names(tmp_path):
    path = tmp_path / "names.txt"
    path.write_text("Ada\nGrace\n", encoding="utf-8")
    names = path.read_text(encoding="utf-8").splitlines()
    assert names == ["Ada", "Grace"]
```

- `tmp_path` provides a separate `pathlib.Path` directory per test.
- Use it for file-based tests instead of a fixed filename in the project.
- This small example illustrates the fixture. Application tests should call the application's file-handling code.

Printed output with capsys

```
def announce(name):
    print(f"Hello, {name}!")

def test_announce(capsys):
    announce("Ada")
    captured = capsys.readouterr()
    assert captured.out == "Hello, Ada!\n"
    assert captured.err == ""
```

capsys captures Python standard output and standard error.
The newline is part of the expected output.

Environment control with monkeypatch

```
# stack.py
import os

def greeting():
    return "Hello, " + os.getenv("USER_NAME", "guest")

# test_resources.py
from stack import greeting

def test_greeting(monkeypatch):
    monkeypatch.setenv("USER_NAME", "Ada")
    assert greeting() == "Hello, Ada"
```

pytest restores the environment change after the test.

Replacing dependencies

- Unit tests can replace a clock, network call, or random source with a controlled substitute.
- `monkeypatch.setattr` temporarily replaces an attribute.
- Patch the name where the code under test looks it up.
- Keep integration tests that exercise the real boundary too.
- A substitute should preserve the relevant interface and behavior. Otherwise a passing test may be misleading.

Floating-point results

```
import pytest

def test_decimal_sum():
    assert 0.1 + 0.2 == pytest.approx(0.3, abs=1e-12,
                                      rel=0)
```

- Binary floating-point arithmetic can introduce rounding error.
- Choose a tolerance that fits the numerical contract.
- Use exact equality for exact results, including integer counts and sets.

Skip and expected failure

```
@pytest.mark.skip(reason="requires the course server")
def test_server():
    ...

@pytest.mark.xfail(strict=True, reason="issue 42: known bug")
def test_known_bug():
    assert buggy_function() == expected
```

- Illustrative examples: the dependency and bug must exist in a real suite.
- `skip` does not run the test. `xfail` normally runs it expecting failure.
- With `strict=True`, an unexpected pass fails the suite so you revisit the marker.

A small language example

Our standalone demo represents expressions with tuples:

```
("var", "x")
("app", function_expr, argument_expr)
("lam", "x", body_expr)
```

The free-variable rules are:

$$FV(x) = \{x\}$$
$$FV(e_1\ e_2) = FV(e_1) \cup FV(e_2)$$
$$FV(\lambda x.e) = FV(e) \setminus \{x\}$$

The same expectations can test a visitor-based implementation.

Exercise: free-variable tests

```
from expr import free_vars

@pytest.mark.parametrize("expr, expected", [
    (("var", "x"), {"x"}),
    (("lam", "x", ("var", "x")), set()),
    # Add an application and a nested lambda.
])
def test_free_vars(expr, expected):
    assert free_vars(expr) == expected
```

- Write a case where a variable remains free inside a lambda.
- Which case catches a lambda rule that forgets to remove its binder?
- Work for five minutes, then compare cases with a neighbor.

Exercise discussion

```
# Application combines both sets.
(("app", ("var", "x"), ("var", "y")), {"x", "y"})

# Binding x leaves y free.
(("lam", "x", ("var", "y")), {"y"})

# Both occurrences of x are bound by the outer lambda.
(("lam", "x", ("app", ("var", "x"),
    ("lam", "y", ("var", "x")))), set())
```

A test of `("lam", "x", ("var", "x"))` catches the missing subtraction.
Set equality checks membership without depending on display order.

A reliable testing workflow

- Describe the intended behavior with a small example.
- Write the test and observe the expected failure.
- Change the implementation and rerun the focused test.
- Run the whole suite before committing the change.
- Keep tests independent of execution order, external state, and wall-clock timing.

What a passing suite tells us

- The implementation satisfies the assertions on the inputs exercised.
- Unasserted behavior and untested cases can still contain bugs.
- Code coverage measures execution, not whether the assertions are strong.
- A useful check: deliberately introduce a plausible bug and see whether a test catches it.

Reference and practice

- Demo: `examples/pytest_demo/`. Run `python -m pytest -q` there.
- Practice: add a stack test that interleaves pushes and pops.
- Practice: add a nested-lambda case with both a bound and a free variable.

Official pytest documentation:

- Getting started and discovery
- Assertions and parametrization
- Fixtures and monkeypatch